

Hunt & Fix: Securing a Web API

OWASP Top 10 (2025) & Peer Code Review — on a real FastAPI app

Course	Units	Work in	Est. time	Submit
Web Systems and Technologies	1 (Laboratory)	Pairs	2–3 hours	GitHub PR links + review sheet

The scenario. Your team inherited **SecureNotes** — a small notes API where users register, log in, and keep private notes. It runs fine and the demo looked great. But it was written in a hurry, and a quick audit says it is riddled with security holes. Before it can ship, someone has to find the holes, fix them, and have a teammate sign off. This week, that someone is you.

What you will practice

- Reading unfamiliar code and **tracing a vulnerability** from an HTTP request to the database.
- Mapping real bugs to the **OWASP Top 10 (2025)** — naming the risk, not just "it's insecure".
- Applying the **secure-coding defences** from the lecture on a live FastAPI + SQLite app.
- Running a genuine **peer code review** — giving and receiving feedback that is specific and kind.

Setup (10 minutes)

You are given the **SecureNotes starter** (`student` folder). Get it running first — a tool you can't run is a tool you can't test.

```
# from inside the student folder
python -m venv venv
source venv/bin/activate # Win:
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Open `http://127.0.0.1:8000/docs` and try `/register`, `/login`, `/notes`.

Seeded accounts (already in the app):

- admin / admin123 (is_admin)
- alice / alicepass
- bob / bobpass

Alice owns note 1, Bob owns note 2. Keep that in mind when you test access control.

Part A — Hunt the vulnerabilities (individually, ~45 min)

There are **exactly 6 security bugs** planted in `app/main.py`. Each one matches a risk from the 2025 Top 10 below. Work through the file and, for every bug you find, fill one row of `FINDINGS.md`: **what** the bug is, **where** it is, **which OWASP risk** it is, and **how an attacker abuses it**.

Tip: don't just read — *attack it*. Log in as Alice, then try to open Bob's note. Send a weird username to `/login`. Call `/admin/users` with a made-up token. The app will tell you a lot.

The six risks in scope — every planted bug is one of these. Use the exact code (A01, A04...) in your findings.

Code	Risk	What it looks like in a web app
A01	Broken Access Control	Reaching data or actions that aren't yours (e.g. another user's record by id).
A02	Security Misconfiguration	Open CORS, hardcoded secrets, stack traces shown to the user.
A04	Cryptographic Failures	Sensitive data (like passwords) stored or sent unprotected.
A05	Injection	Untrusted input glued into a query so it runs as code (SQL injection).
A07	Authentication Failures	Guessable or never-expiring tokens; login that leaks which accounts exist.
A10	Exceptional Conditions (new 2025)	A failed check that "fails open" and allows the action anyway.

Part B — Fix them (as a pair, ~60 min)

Now make `app/main.py` secure. Two rules:

1. **Close every hole** you listed in Part A. Re-run your attack afterwards to prove it no longer works.
2. **Don't break the app.** Register, login, and listing your own notes must still work at the end.

You only need FastAPI and the Python standard library — `hashlib`, `hmac`, `secrets`, `time`. (Proper JWT auth comes in Week 14; a simple signed, expiring token is fine here.)

Fix cheat-sheet — the habits from the lecture, in order of "do this first":

- **Injection** → use a **parameterized query** (`WHERE username = ?`), never an f-string.
- **Passwords** → **salt + hash** (PBKDF2 / bcrypt); compare with `hmac.compare_digest`.
- **Access** → check the record **belongs to the caller** before returning it.
- **Auth** → make tokens **signed and expiring**; return one **generic** login error.
- **Config** → lock CORS to real origins, move the secret to an **env var**, stop leaking stack traces.
- **Exceptions** → make permission checks **fail closed** — deny when anything goes wrong.

Part C — Peer code review on GitHub (~40 min)

Real teams review code on GitHub through a **pull request (PR)** — the same flow from the lecture. You'll push your project, open a PR that shows your security fixes as a clean diff, and review another pair's PR.

C1 · Push your project and open a PR (author pair)

Create a **public** repo on GitHub (e.g. `securenotes-w13`). Commit the original starter first, then your fixes on a branch — so the PR diff shows exactly what you changed and why.

```
# in your project folder, BEFORE you start fixing (or on a fresh copy of the starter)
git init
git add . && git commit -m "Starter: SecureNotes (vulnerable)"
git branch -M main
git remote add origin https://github.com/<your-username>/securenotes-w13.git
git push -u origin main

# now put your fixes on their own branch
git checkout -b security-fixes
# ... apply your fixes + fill FINDINGS.md ...
```

```
git add . && git commit -m "Fix OWASP Top 10 issues (A01, A02, A04, A05, A07, A10)"
git push -u origin security-fixes
```

On GitHub, click **Compare & pull request** → base `main`, compare `security-fixes`. In the PR description, list the 6 bugs and their OWASP codes. Then share the **PR link** with your assigned reviewer pair.

C2 · Review your partner pair's PR (reviewer pair)

Open their PR, read the **Files changed** tab, and clone the repo to run it. Leave your feedback as **line comments on the diff** (click the `+` on a line), then finish with **Review changes** → **Approve** or **Request changes**.

Review checklist (also on the last page)

- ☐ All 6 risks found and correctly named?
- ☐ Each fix actually blocks the attack (you tested it)?
- ☐ Register / login / notes still work?
- ☐ No passwords stored or returned in plain text?
- ☐ No secret or stack trace left in the code?
- ☐ Permission checks fail *closed*, not open?

Your PR comments must be...

- **Specific** — comment on the exact line; name the risk.
- **Kind** — about the code, never the person.
- **Actionable** — suggest a direction, phrased as a question.

Leave at least **three** line comments: one strength, one must-fix, one suggestion — then Approve or Request changes.

New to Git? Install Git, then make a free account at github.com. If the push asks for a password, use a **Personal Access Token** (GitHub → Settings → Developer settings → Tokens), not your account password. Keep the repo small — do **not** commit your `venv/` folder (add a `.gitignore` with `venv/` and `__pycache__/`).

Deliverables

1. **Your GitHub repo link** — fixed code on the `security-fixes` branch, with `FINDINGS.md` in it.
2. **Your Pull Request link** — `security-fixes` → `main`, description listing the 6 bugs + OWASP codes.
3. **The PR you reviewed** — link to your partner pair's PR showing your line comments and your Approve / Request-changes.
4. **Peer Review Sheet** (below) — a short summary of the review you gave.

Submit the **two links** (your PR + the PR you reviewed) and the review sheet through the LMS. One submission per pair.

Rubric (100 points)

Criteria	What we look for	Pts
Finding & naming	All 6 bugs identified and mapped to the correct OWASP code in FINDINGS.md.	30
Fixes that work	Each attack is blocked after the fix (we re-run the exploits).	35
App still works	Register, login, and listing your own notes all function.	10
Peer review quality	Checklist complete; 3+ specific, constructive comments.	15
Reflection	Thoughtful answer on which bug is most dangerous and why.	10

Peer Review Sheet

Reviewer pair: _____ Code authored by: _____ Date: _____

Check	Pass?	Note
All 6 risks found & correctly named	Y / N	
Every fix blocks its attack (tested)	Y / N	
App still works (register/login/notes)	Y / N	
No plaintext passwords stored/returned	Y / N	
No secret / stack trace left in code	Y / N	
Permission checks fail closed	Y / N	

1 strength: _____

1 must-fix: _____

1 suggestion: _____

This lab brings both halves of Week 13 together: you apply the OWASP defences to real code, then you practise the review habits that keep a whole team's code safe. Next week (Week 14) you'll rebuild this app's login properly with JWT authentication.